

CS112: Monday 5 October, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM W05-CONTAINERS AND ERIC'S ACCOUNT OF THE CLASS

Friday we finished the rule of three. `PyList` has a destructor, a copy constructor and an `operator=`, so a copy owns its own block and nothing leaks. The class works, for one type.

Today: a `PyList` of `int` and a `PyList` of `string`, in one program. `typedef` cannot do it. The fix is a class template, and everything was typed live in `w05-containers`, with the tests written first.

PART

Daily quiz

1. Two lists, two types
2. The test we want to pass
3. `template <typename Item>`
4. Converting the class, and losing the `.cpp`
5. `vector`, already written
6. Close

Part 1. Two lists, two types

Talk to your neighbor. We want a `PyList` of `int` and a `PyList` of `string` in the same program. What do we change?

`typedef` is last week's tool, and it is the one that fails here.

```
typedef int Item;
```

That names one type for the whole file. A second `typedef` for `string` is a redefinition and the compiler says so. Copying `PyList.h` into `PyListString.h` does work, and it is the reason templates exist: two files, one idea, every fix made twice.

Three places in the class actually mention the type. The member `Item *myArray`, the parameter of `append`, and the return type of `getIndex` and `operator[]`. Everything else is the same in both copies.

Part 2. The test we want to pass

w05-containers/tests.cpp gets written before a single `template` keyword does.

```
TEST_CASE("template class"){
    PyList<string> students;
    students.append("Kwesi");
    REQUIRE(students.getSize() == 3);
    REQUIRE(students[0] == "Kwesi");
}
```

This does not compile. `PyList<string>` is not a thing yet, and that is the point: the test says what the class has to become. The older sections stay exactly as they were, with `PyList<int>` in place of `PyList`, so the conversion has to keep capacity doubling, bounds throwing and the copy constructor all working.

Red first, then the class.

Part 3. `template <typename Item>`

```
template<typename Item>
class PyList{
    ...
private:
    Item *myArray;
    int mySize;
    int myCapacity;
};
```

`Item` is a parameter, and what it parameterizes is the class itself.

Talk to your neighbor. A program uses `PyList<int>` and `PyList<string>`. How many classes does the compiler build from this one file?

Two. Each is generated separately, compiled on its own, and they are unrelated types. Neither can be assigned to the other. A template is a blueprint, not a class, which is why `PyList` on its own is no longer the name of a type.

Part 4. Converting the class, and losing the .cpp

Every method definition picks up the same two pieces of ceremony.

```
template<typename Item>
void PyList<Item>::append(Item it) {
```

The parameter is declared again above each one, and the class name carries `<Item>` from there on. The copy constructor is the exception. Inside the template, the bare `PyList` in `const PyList &orig` still means this one.

Then `PyList.cpp` has to disappear. A template is a blueprint, so the compiler can only build `PyList<string>` where it can see the body of every method, and a separately compiled `.cpp` hides exactly that. Every definition moves into `PyList.h`.

The makefile follows on its own. `SRCS := $(wildcard *.cpp)` now finds only `tests.cpp`, and the rule that makes every object depend on the headers is what rebuilds the tests when `PyList.h` changes.

`make test` is green, with `PyList<int>`, `PyList<string>` and the copy constructor all exercised.

Part 5. `vector`, already written

The standard library ships containers, and the one that matches what we just built is `vector`.

```
#include <vector>

vector<string> names;
names.push_back("Micaiah");
names.push_back("Cayden");
names.push_back("Ana Clara");
names.push_back("Diego");

assert(names[3] == "Diego");
assert(names.size() == 4);
```

`vector<string>` is a class generated from a class template, exactly as `PyList<string>` now is. The same angle brackets, for the same reason.

Both asserts pass, which is the whole demo. How you walk one without an index is Wednesday.

Part 6. Close

Our container takes a type parameter now, and the standard library has one already written.

Wednesday: `begin()`, `end()`, and what past the end means.